

RAG + LLM Flow With A Real Example

Project: eCOM-AI | Example question: Which delivery option is cheapest?

Correct flow: The app does not send the user question directly to the LLM. It first detects intent, retrieves filtered store data, checks confidence and source coverage, applies structured ranking, and only then asks the LLM to write the final answer from verified sources.

Full Flow

```
User question
-> AiChatService detects preferredSourceType
-> AiChatService detects maximumPrice
-> AiRetrieverService searches using question + filters
-> Retrieved results are confidence-checked
-> Results are reordered for cheapest / most expensive
-> Source coverage check verifies support
-> LLM writes final answer using verified sources only
```

Step 1: User Asks A Question

The customer asks:

```
Which delivery option is cheapest?
```

This question contains two important signals:

- delivery option means the app should search delivery/policy documents.
- cheapest means the app should use actual price metadata for ranking.

Step 2: Detect Preferred Source Type

AiChatService checks the words in the question.

Because the question contains delivery , the service sets:

```
preferredSourceType = "policy"
```

This matters because delivery questions should not be answered from product documents.

Real Meaning

Question Signal	Detected Intent	Effect
delivery	policy	Search delivery policy documents only.

Step 3: Detect Maximum Price

The app also checks whether the question contains a phrase like:

- under 20
- below 20
- less than 20
- cheaper than 20

Our example question does not give a price limit, so:

```
maximumPrice = null
```

That means no price filter is applied during retrieval. Price will still be used later for cheapest ranking.

Step 4: Retrieve Using Question + Filters

aiRetrieverService receives:

```
question = "Which delivery option is cheapest?"
preferredSourceType = "policy"
maximumPrice = null
```

It searches the RAG index or in-memory fallback and only keeps policy/delivery documents.

Example retrieved sources:

Retrieved Source	Type	Price	Estimated Time
UPS3 delivery	policy	\$2.55	5-10 Days
FREE delivery	policy	\$0.00	1-2 Weeks
UPS2 delivery	policy	\$5.99	2-5 Days

Note: Retrieval finds relevant documents, but retrieval alone does not decide which one is cheapest.

Step 5: Confidence Check

The app checks whether the retrieved results are strong enough to answer.

If the top score is too weak, it stops and returns a safe fallback instead of answering confidently.

```
if topScore < minimumRequiredScore:
    return "I found some related information, but not enough to answer confidently."
```

For this example, the delivery sources are strong enough, so the flow continues.

Step 6: Structured Ranking For Cheapest

This is the key fix for the cheapest-delivery issue.

Because the question contains `cheapest` , the app sorts retrieved matches by real numeric price:

```
OrderBy(price)
ThenByDescending(AI similarity score)
ThenBy(title)
```

Before structured ranking, vector similarity might put `UPS3 delivery` first because its description says "cheap". But actual price proves `FREE delivery` is cheapest.

Source	Price	Rank After Structured Sorting
FREE delivery	\$0.00	1
UPS3 delivery	\$2.55	2
UPS2 delivery	\$5.99	3

Step 7: Source Coverage Check

The app checks whether the sources actually support the question.

For this question, the sources include delivery options and prices, so the app can answer.

But if the question were:

```
Do you offer cash on delivery?
```

The app would reject it unless the retrieved sources mention `cash` , `COD` , or `payment` . This prevents the LLM from guessing.

Step 8: LLM Writes The Final Answer

- Only now does the app call the LLM.
- The LLM receives:
- The original question.
 - The verified sources.
 - Instructions to use only those sources.
 - Instructions not to invent products, prices, policies, discounts, warranties, locations, or payment methods.

Final answer from the live test:

```
The cheapest delivery option is FREE delivery, which costs $0.00.
```

Why This Is Real RAG

This is RAG because the LLM does not answer from memory. The app retrieves store data first, validates it, and then gives that verified context to the LLM.

Part	Responsibility
Intent detection	Decides whether to search products or policy/delivery data.
Retriever	Finds relevant store documents.
Structured filters/ranking	Uses exact metadata like price.
Source coverage check	Blocks unsupported answers.
LLM	Turns verified sources into a clear customer-facing answer.

Another Quick Example: Products Under \$20

```
Question:
What products are available under 20 dollars?

Detected preferredSourceType:
product

Detected maximumPrice:
20

Retriever uses:
question + sourceType product + price < 20

LLM answer from live test:
Products under $20 found in the store data:
- Green Code Gloves - $15.00
- Blue Code Gloves - $18.00
- Green React Gloves - $14.00
```

Final Takeaway

The LLM is the writer, not the source of truth. The source of truth is still the database/search documents and their metadata. That is what keeps the assistant accurate.